Loops in JavaScript

while, for, and do-while -- when to use each and how they work

Loops let you run the same block of code over and over until some condition tells them to stop. They're one of the most fundamental tools you have as a programmer. JavaScript gives you three main flavors, and each one has its sweet spot. Let's walk through all three.

The three loop types at a glance:
while -- checks the condition first, then runs. Best when you don't know how many times you'll loop.
for -- everything in one line (init, condition, update). Best when you know exactly how many times to loop.
do-while -- runs first, then checks the condition. Guarantees at least one execution.

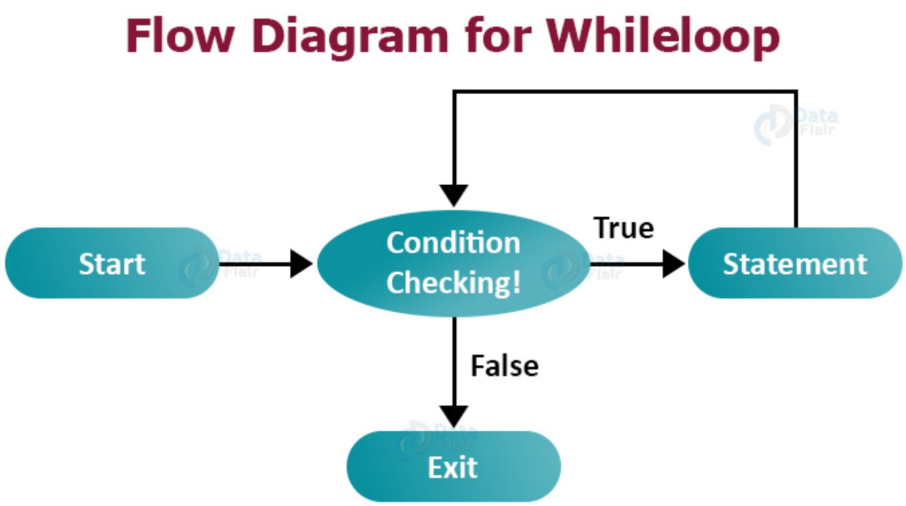
Before we dive in, every loop has four building blocks: **initialization** (where do we start?), **condition** (should we keep going?), the **body** (what do we do each time?), and **increment/decrement** (how do we move forward?). The three loop types just arrange these pieces differently.

---

1. The while Loop

The while loop checks its condition before each iteration. If the condition is false from the start, the code inside never runs at all. You set up your counter variable outside the loop, and you're responsible for updating it inside.

```
while (condition) {
    // code runs as long as condition is true
}
```



Flow Diagram: while loop -- condition is checked BEFORE the statement executes

Print numbers 1 to 10:

```
let i = 1;              // initialization (outside the loop)
while (i <= 10) {       // condition checked before each run
    console.log(i);     // prints: 1 2 3 4 5 6 7 8 9 10
    i++;                // increment (your responsibility!)
}
```

Breaking out early with break:

Sometimes you want to bail out of a loop before the condition naturally becomes false. That's what `break` is for.

```
let j = 1;
while (j <= 10) {
    console.log(j);        // prints: 1 2 3 4 5
    if (j === 5) {
        console.log('bye');
        break;              // stops the loop right here
    }
    j++;
}
```

Infinite loops (controlled):

An infinite loop runs forever unless you manually break out of it. This sounds scary, but it's actually useful in real-world scenarios like waiting for something to happen or continuously checking a condition.

```
// Using while(true) with a controlled exit
let k = 1;
while (true) {
    console.log('waiting...');
    if (k === 5) {
        console.log('done!');
        break;               // always have an exit!
    }
    k++;
}
```

Warning: If you forget the break or forget to update your counter, you'll get an actual infinite loop that freezes your browser or crashes your program. Always double-check that your loop has a way to end.

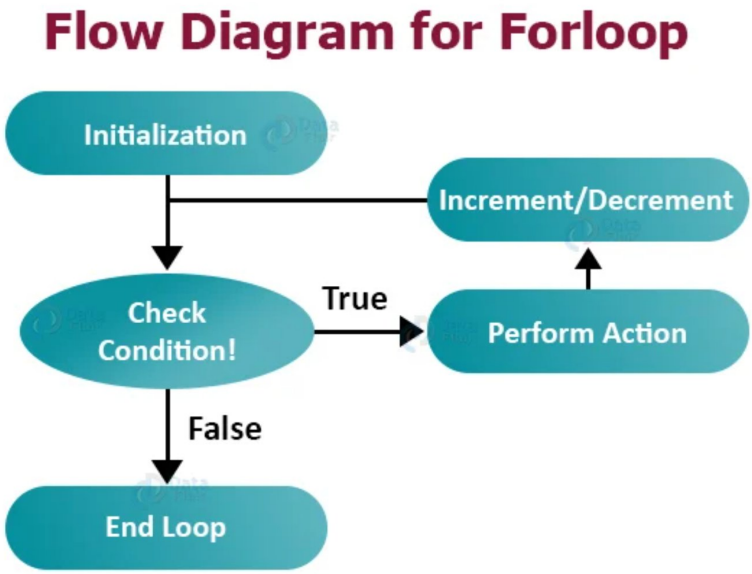
When to reach for a while loop:

- You don't know how many times you'll need to loop
- Waiting for web elements to load (polling every few seconds)
- Page loading, dynamic loaders, or progress bars (0 to 100%)
- Web table pagination -- keep clicking 'Next' until there are no more pages
- Infinite scrolling pages -- keep scrolling until content stops loading
- Processing an unknown number of links or elements on a page
- Calendar and carousel handling
- Database queries -- read rows until you hit the end
- File downloading/uploading progress
- Waiting for dynamic alerts or popups to appear

2. The for Loop

The for loop packs initialization, condition, and increment all into one line. It's more compact and harder to mess up because everything's right there in front of you. This is your go-to when you know exactly how many times to iterate.

```
for (initialization; condition; increment/decrement) {
    // code to be executed
}
```



Flow Diagram: for loop -- initialization happens once, then condition > action > increment repeats

Print numbers 1 to 10:

```
for (let m = 1; m <= 10; m++) {
    console.log(m);     // prints: 1 2 3 4 5 6 7 8 9 10
}

// Compare this with the while version -- same result,
// but everything is on one line. Cleaner.
```

You can also split things up:

Technically, all three parts of the for loop are optional. You can move them outside if you need to, though this is less common:

```
let n = 1;
for (; n <= 10;) {      // init and increment moved outside
    console.log(n);
    n++;
}

// And if you leave everything empty, you get an infinite loop:
for (;;) {
    console.log('infinite!');
    break;               // don't forget the exit!
}
```

Counting backwards:

```
for (let s = 10; s >= 1; s--) {
    console.log(s);     // prints: 10 9 8 7 6 5 4 3 2 1
}
```

Even and odd numbers:

```
// Even numbers from 1 to 10
for (let num = 1; num <= 10; num++) {
    if (num % 2 === 0) {
        console.log(num);   // prints: 2 4 6 8 10
    }
}

// Odd numbers from 1 to 10
for (let num = 1; num <= 10; num++) {
    if (num % 2 === 1) {
        console.log(num);   // prints: 1 3 5 7 9
    }
}
```

When to reach for a for loop:

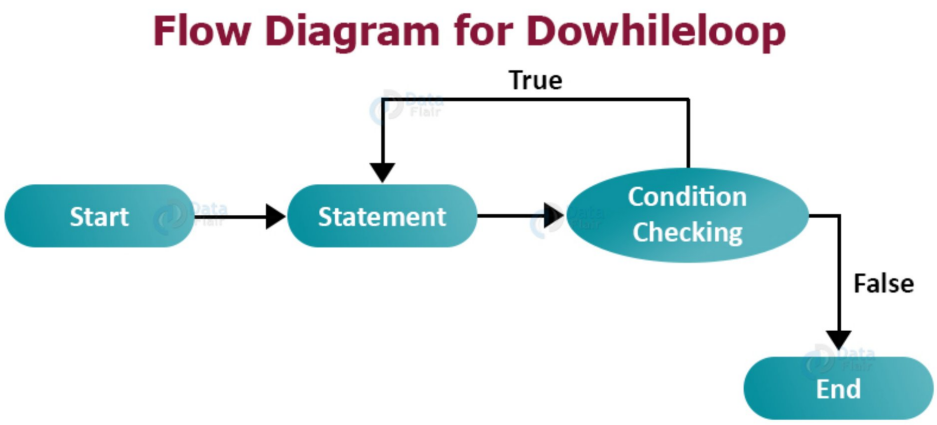
- You know exactly how many iterations you need
- Dropdown menus with fixed options (months, days, countries)
- Processing Excel/CSV files with a known number of rows
- Iterating through arrays or lists with a known length
- Seat booking systems (fixed number of seats in a theater, flight, bus)
- Menu items, navigation links, or footer links
- Login attempt limits (e.g., max 3 tries before lockout)
- Retry logic for failed test cases (try up to N times)

Rule of thumb: If you find yourself writing a while loop with a simple counter that starts at X and ends at Y, a for loop is almost always cleaner. Save while loops for when the number of iterations is genuinely unknown.

3. The do-while Loop

The do-while loop is the oddball of the three. It runs the code first and checks the condition after. This means the body always executes at least once, even if the condition is false from the very beginning. It's the least commonly used loop, but it has its moments.

```
do {
    // code runs at least once
} while (condition);    // note the semicolon!
```



Flow Diagram: do-while loop -- statement runs FIRST, then condition is checked

Print numbers 1 to 10:

```
let p = 1;
do {
    console.log(p);     // prints: 1 2 3 4 5 6 7 8 9 10
    p++;
} while (p <= 10);
```

Runs at least once -- even when the condition is false:

```
do {
    console.log('this runs once!');  // executes!
} while (false);
// The condition is false, but the body already ran.

// Compare with a while loop:
while (false) {
    console.log('this never runs');  // never executes
}
```

Another example showing the post-test behavior:

```
let d = 1;
do {
    console.log(d);     // prints: 1
    d++;                // d becomes 2
} while (d >= 10);      // 2 >= 10 is false, so loop stops

// Only printed once because the condition failed after
// the first execution.
```

When to reach for a do-while loop:

- When the code must execute at least once before checking any condition
- Landing on a page and immediately checking if an element exists
- Initial setup or validation that must happen before condition checking
- User input prompts that need to show at least once (like "Enter password")

Side-by-Side Comparison

Feature	while	for	do-while
Condition check	Before each run	Before each run	After each run
Min executions	0 (might never run)	0 (might never run)	1 (always runs once)
Best for	Unknown iterations	Known iterations	Must-run-once logic
Counter setup	Outside the loop	Inside the loop header	Outside the loop
Readability	Good for dynamic loops	Most compact / clean	Can be confusing
Common in testing	Waits, polling, scrolling	Arrays, retries, fixed lists	Initial checks

Key Concepts to Remember

- **Initialization** -- setting the starting value of your loop variable (e.g., let i = 0)
- **Condition** -- the boolean check that decides whether the loop keeps running
- **Increment/Decrement** -- how the loop variable changes after each iteration (i++, i--, i+=2, etc.)
- **break** -- immediately exits the loop, no questions asked
- **continue** -- skips the rest of the current iteration and jumps to the next one

Avoiding infinite loops: Always make sure your condition will eventually become false. The most common mistakes are forgetting to increment your counter, or writing a condition that can never be false (like `while (i > 0)` when i starts at 1 and only increases). If your program hangs, an accidental infinite loop is usually the first thing to check.

Practice Exercises

Try these on your own to get comfortable with all three loop types:

1. Write a while loop to find the sum of all numbers from 1 to 100.
2. Create a for loop to print the multiplication table of 5 (5x1=5, 5x2=10, ... 5x10=50).
3. Use a do-while loop to keep asking for user input until they type "quit".
4. Write a loop that counts how many vowels are in a given string.
5. Create a loop that reverses a number (e.g., 12345 becomes 54321).

Loops are one of those things that become second nature with practice. Start with for loops for simple counting, graduate to while loops when things get dynamic, and keep do-while in your back pocket for those "run at least once" situations.